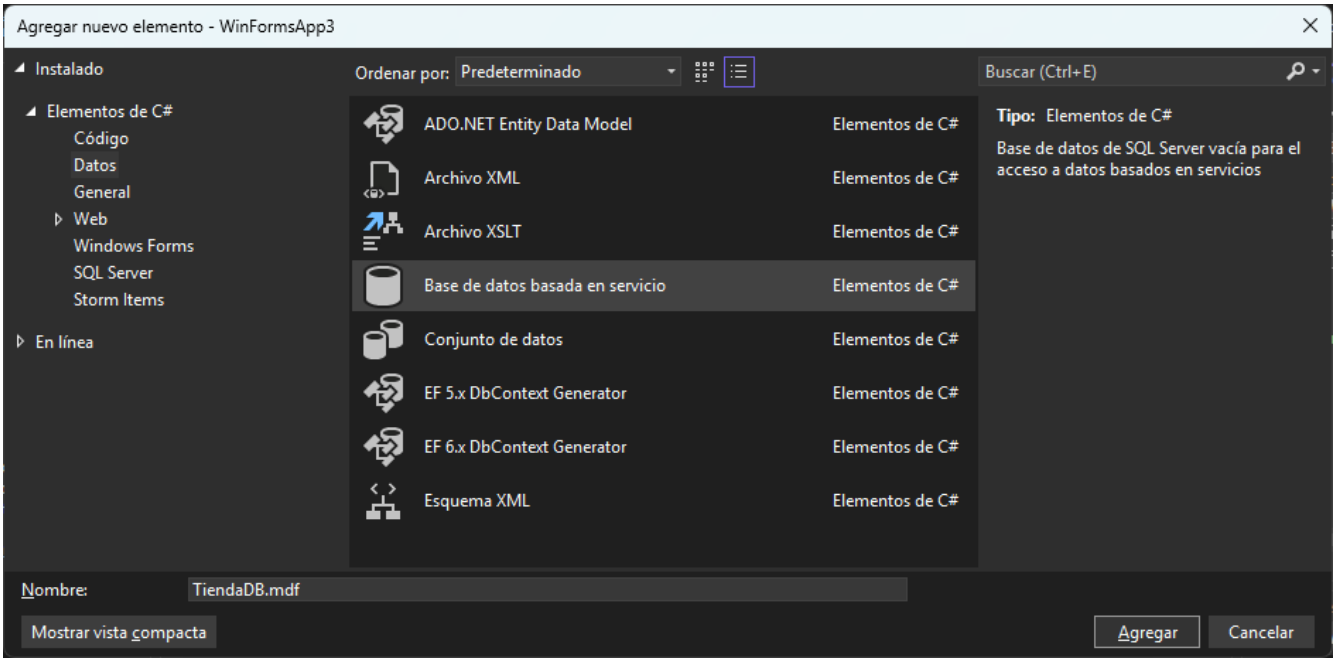
CRUD EN WINDOWS FORMS

Para crear un **CRUD** (Crear, Leer, Actualizar, Eliminar) en **Windows Forms** con **Visual Studio 2022** utilizando una base de datos local **.mdf** (SQL Server Express LocalDB) con **varias tablas**, la forma más rápida, eficiente y mantenible es utilizar **Entity Framework Core** (EF Core). A continuación, el paso a paso para un escenario clásico de dos tablas relacionadas: `Categorias` y `Productos` (Relación de uno a muchos).

Paso 1: Configurar el Proyecto y la Base de Datos

1. Abrir **Visual Studio 2022** y crear un nuevo proyecto de tipo **Aplicación de Windows Forms (.NET Desktop)** seleccionando **.NET 8.0** o **.NET 6.0**.
2. Haga clic derecho sobre el proyecto en el Explorador de soluciones → **Agregar** → **Nuevo elemento...**
3. Busque **Base de datos basada en servicios** (archivo .**mdf**), nómbrala **`TiendaDB.mdf`** y dele clic al botón Agregar.



4. Abra el **Explorador de servidores**, dele doble clic a su base de datos, haga clic derecho en **Tablas** → **Agregar nueva tabla** y crea las siguientes dos tablas:

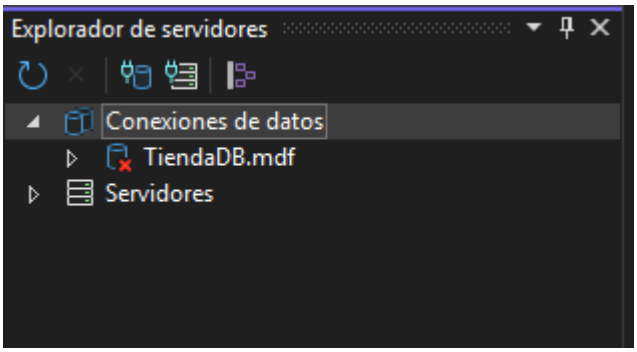


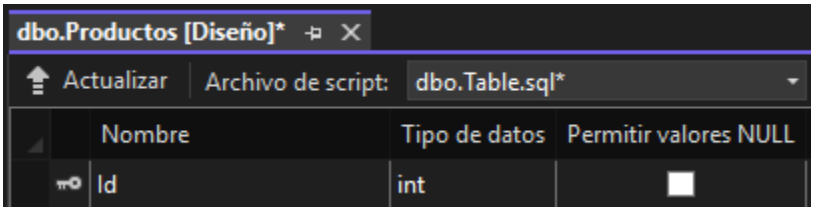
Tabla: Categorias

```
CREATE TABLE [dbo].[Categorias] (
    [Id]INT IDENTITY (1, 1) NOT NULL,
    [Nombre] NVARCHAR (50) NOT NULL,
    PRIMARY KEY CLUSTERED ([Id] ASC)
);
```

Tabla: Productos

```
CREATE TABLE [dbo].[Productos] (
    [Id] INT IDENTITY (1, 1) NOT NULL,
    [Nombre] NVARCHAR (100) NOT NULL,
    [Precio] DECIMAL (18, 2) NOT NULL,
    [CategoriaId] INT NOT NULL,
    PRIMARY KEY CLUSTERED ([Id] ASC),
    CONSTRAINT [FK_Productos_Categorias] FOREIGN KEY ([CategoriaId])
REFERENCES [dbo].[Categorias] ([Id]) ON DELETE CASCADE
);
```

*Haga clic en el botón **Update** (Actualizar) en el diseñador de SQL para aplicar los cambios a cada tabla en la Base de Datos.*



Paso 2: Instalar Paquetes de Entity Framework Core

Abra la **Consola del Administrador de Paquetes** (Herramientas → Administrador de paquetes NuGet → Consola...) y ejecuta los siguientes comandos para instalar el soporte de SQL Server:

```
Install-Package Microsoft.EntityFrameworkCore.SqlServer
```

Paso 3: Crear el Modelo y el Contexto de Datos (Código)
Cree una carpeta llamada `Models` y agregue las siguientes tres clases:

1. Categoría (`Categoria.cs`)

```
using System.Collections.Generic;

namespace SuProyecto.Models
{
    public class Categoria
    {
        public int Id { get; set; }
        public string Nombre { get; set; } = null!;

        // Relación: Una categoría tiene muchos productos
        public virtual ICollection<Producto> Productos { get; set; } = new List<Productos>();
    }
}
```

2. Producto (`Producto.cs`)

```
namespace TuProyecto.Models
{
    public class Producto
    {
        public int Id { get; set; }
        public string Nombre { get; set; } = null!;
        public decimal Precio { get; set; }
        public int CategoriaId { get; set; }

        // Propiedades de navegación
        public virtual Categoria Categorias { get; set; } = null!;
    }
}
```

3. Contexto de Base de Datos (`TiendaContext.cs`)

```
using Microsoft.EntityFrameworkCore;
using System.IO;

namespace SuProyecto.Models
{
    public class TiendaContext : DbContext
    {
        public DbSet<Categoria> Categorias { get; set; }
        public DbSet<Producto> Productos { get; set; }

        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            // Ruta automatizada hacia tu archivo MDF local
            string baseDir = AppDomain.CurrentDomain.BaseDirectory;
            string dbPath = Path.Combine(baseDir, "TiendaDB.mdf");

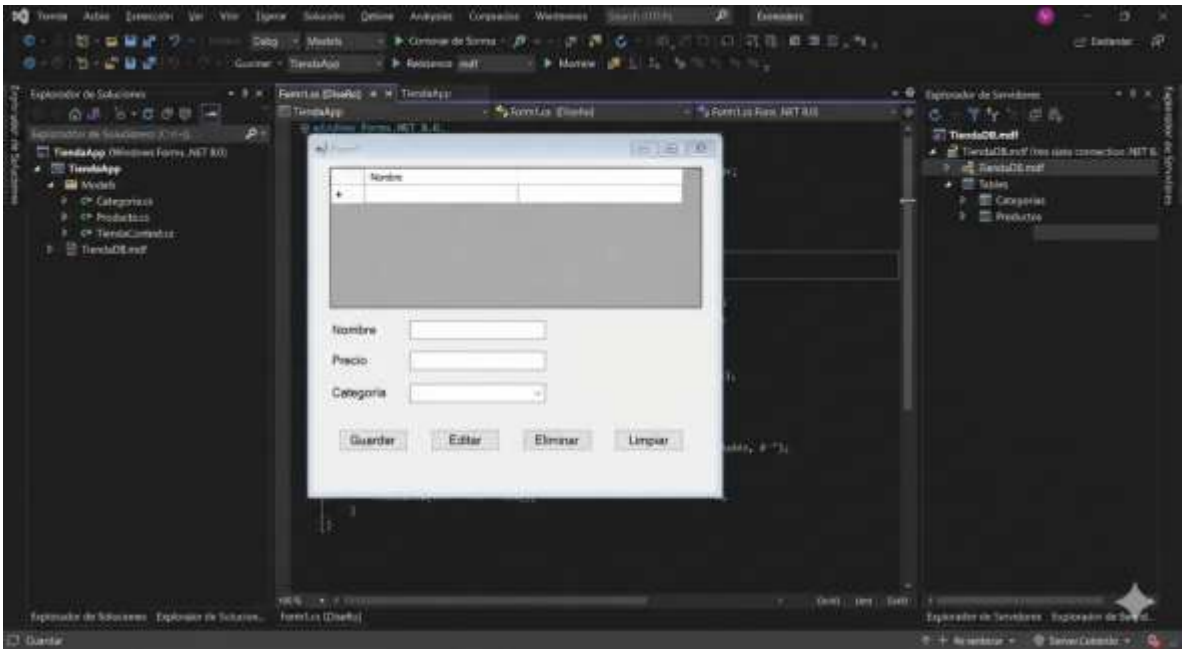
            optionsBuilder.UseSqlServer($@"Data
Source=(LocalDB)\MSSQLLocalDB;AttachDbFilename={dbPath};Integrated
Security=True;TrustServerCertificate=True");
        }
    }
}
```

⚠️ **Nota Importante:** Asegúrese de que las propiedades del archivo `TiendaDB.mdf` tengan configurado **"Copiar en el directorio de salida"** como **"Copiar si es posterior"** para que se mueva correctamente junto al ejecutable.

Paso 4: Diseñar la Interfaz en Windows Forms (`Form1.cs`)

Diseñe un formulario que le permita gestionar los Productos y asignarles una Categoría. Agregue los siguientes controles al lienzo del formulario:

- ✓ **DataGridView** (`dgvProductos`): Para listar los productos.
- ✓ **TextBox** (`txtNombre`): Para el nombre del producto.
- ✓ **TextBox** (`txtPrecio`): Para el precio.
- ✓ **ComboBox** (`cmbCategorias`): Para seleccionar la categoría (llave foránea).
- ✓ **4 Botones**: `btnGuardar`, `btnEditar`, `btnEliminar`, `btnLimpiar`.



Paso 5: Escribir la Lógica del CRUD en el Formulario

Reemplace el código de su Form1.cs con la siguiente lógica:

```
using System;
using System.Linq;
using System.Windows.Forms;
using Microsoft.EntityFrameworkCore;
using SuProyecto.Models;

namespace TuProyecto
{
    public partial class Form1 : Form
    {
        private TiendaContext _context = new TiendaContext();
        private int _productoIdSeleccionado = 0;

        public Form1()
        {
            InitializeComponent();
        }

        private void Form1_Load(object sender, EventArgs e)
        {
```

```
        CargarCategorias();
        CargarProductos();
    }

    // --- MÉTODOS DE SOPORTE ---
    private void CargarCategorias()
    {
        // Insertar datos de prueba en categorías si está vacío
        if (!_context.Categorias.Any())
        {
            _context.Categorias.AddRange(
                new Categoria { Nombre = "Electrónica" },
                new Categoria { Nombre = "Ropa" }
            );
            _context.SaveChanges();
        }

        cmbCategorias.DataSource = _context.Categorias.ToList();
        cmbCategorias.DisplayMember = "Nombre";
        cmbCategorias.ValueMember = "Id";
    }

    private void CargarProductos()
    {
       // READ: Carga productos incluyendo los datos de la tabla relacionada Categorias
        var productos = _context.Productos
            .Include(p => p.Categoria)
            .Select(p => new
            {
                p.Id,
                Producto = p.Nombre,
                p.Precio,
                Categoria = p.Categoria.Nombre,
                p.CategoriaId
            }).ToList();

        dgvProductos.DataSource = productos;
        // Ocultar columna de ID de categoría
        if (dgvProductos.Columns["CategoriaId"] != null)
            dgvProductos.Columns["CategoriaId"].Visible = false;
    }
```

```
private void LimpiarFormulario()
{
    txtNombre.Clear();
    txtPrecio.Clear();
    cmbCategorias.SelectedIndex = 0;
    _productoIdSeleccionado = 0;
    btnGuardar.Text = "Guardar";
}

// --- ACCIONES CRUD ---

// CREATE / UPDATE
private void btnGuardar_Click(object sender, EventArgs e)
{
    if (string.IsNullOrEmpty(txtNombre.Text) ||
        !decimal.TryParse(txtPrecio.Text, out decimal precio))
    {
        MessageBox.Show("Por favor, rellene los campos correctamente.");
        return;
    }

    if (_productoIdSeleccionado == 0)
    {
        // CREATE
        var nuevoProducto = new Producto
        {
            Nombre = txtNombre.Text,
            Precio = precio,
            CategoriaId = (int)cmbCategorias.SelectedValue
        };
        _context.Productos.Add(nuevoProducto);
    }
    else
    {
        // UPDATE
      var productoExistente = _context.Productos.Find(_productoIdSeleccionado);
        if (productoExistente != null)
        {
            productoExistente.Nombre = txtNombre.Text;
            productoExistente.Precio = precio;
```

```
                productoExistente.CategoriaId = (int)cmbCategorias.SelectedValue;
            }
        }

        _context.SaveChanges();
        CargarProductos();
        LimpiarFormulario();
        MessageBox.Show("Operación realizada con éxito.");
    }

    // Seleccionar registro para editar desde el DataGridView
    private void dgvProductos_CellClick(object sender, DataGridViewCellEventArgs e)
    {
        if (e.RowIndex >= 0)
        {
            DataGridViewRow row = dgvProductos.Rows[e.RowIndex];
            _productoIdSeleccionado = Convert.ToInt32(row.Cells["Id"].Value);
            txtNombre.Text = row.Cells["Producto"].Value.ToString();
            txtPrecio.Text = row.Cells["Precio"].Value.ToString();
            cmbCategorias.SelectedValue =
                        Convert.ToInt32(row.Cells["CategoriaId"].Value);

            btnGuardar.Text = "Actualizar";
        }
    }

    // DELETE
    private void btnEliminar_Click(object sender, EventArgs e)
    {
        if (_productoIdSeleccionado == 0)
        {
            MessageBox.Show("Seleccione un producto de la lista para eliminar.");
            return;
        }

        var confirm = MessageBox.Show("¿Está seguro de eliminar este
                    producto?", "Confirmar", MessageBoxButtons.YesNo);

        if (confirm == DialogResult.Yes)
        {
            var producto = _context.Productos.Find(_productoIdSeleccionado);
```

```
                if (producto != null)
                {
                    _context.Productos.Remove(producto);
                    _context.SaveChanges();
                    CargarProductos();
                    LimpiarFormulario();
                    MessageBox.Show("Producto eliminado.");
                }
            }
        }

        private void btnLimpiar_Click(object sender, EventArgs e)
        {
            LimpiarFormulario();
        }
    }
}
```

Ventajas de este enfoque multitabla

1. **Mantenimiento**: No escribe código SQL crudo (INSERT INTO...), lo que previene errores tipográficos.
2. **Relaciones automáticas**: Gracias a `.Include(p => p.Categoria)`, EF Core se encarga de hacer el INNER JOIN tras bambalinas automáticamente.
3. **Escalabilidad**: Si añade una tercera tabla (ej. Proveedores), solo crea el modelo y lo agrega al contexto `TiendaContext`.

SUBIENDO A GITHUB EL PROYECTO

Para subir su proyecto de Windows Forms a **GitHub** desde **Visual Studio 2022**, puede hacerlo de forma directa y visual sin necesidad de usar comandos en la terminal. Siga estos pasos secuenciales para configurar su repositorio y subir el código de manera limpia:

1. Preparar el archivo `.gitignore` (Muy importante para bases de datos MDF)
Antes de subir el proyecto, debe asegurarse de que los archivos temporales de compilación y los bloqueos de la base de datos local (archivos `.ldf` o temporales de SQL Server) no se suban a la nube. Visual Studio maneja esto de forma automática al crear el repositorio.

2. Pasos dentro de Visual Studio 2022

1. **Abrir la herramienta de Git:**
   - En la esquina inferior derecha de Visual Studio, haga clic en el botón **Agregar al control de código fuente** (Add to Source Control) y seleccione **Git**.
   - *Alternativa:* Vaya al menú superior **Git** → **Crear repositorio Git...**

2. **Configurar su cuenta de GitHub:**
   - En la ventana que se abre, asegúrese de que esté seleccionada la opción **GitHub** en el panel izquierdo.
   - Si no ha iniciado sesión, haga clic en **Iniciar sesión** (Sign in) para vincular su cuenta de GitHub con Visual Studio.

3. **Definir los datos del repositorio:**
   - **Propietario:** Su cuenta de GitHub.
   - **Nombre del repositorio:** Por defecto tomará el nombre de su proyecto (ej. TiendaApp).
   - **Descripción:** (Opcional) Puede escribir algo como *"CRUD multitabla en C# con Windows Forms y base de datos MDF"*.
   - **Repositorio privado:** Desmarque esta casilla si quiere que su código sea **público** y otros puedan verlo, o déjela marcada si prefiere mantenerlo privado.

4. **Crear y subir (Push):**
   - Haga clic en el botón **Crear e insertar** (Create and Push).
   - Visual Studio creará un repositorio local, generará el archivo `.gitignore` adecuado para proyectos de .NET, hará el primer commit con todo su código y lo subirá directamente a su cuenta de GitHub.

3. Verificar el resultado

- Entre a su perfil de GitHub desde el navegador.
- En la sección de **Repositories**, verá su nuevo proyecto recién subido con toda la estructura de carpetas (Models, archivos .cs, etc.).

4. ¿Cómo subir futuros cambios? (El flujo diario)

Cada vez que haga modificaciones en sus formularios o en el contexto de datos, siga estos **3 pasos rápidos** en la pestaña **Cambios de Git** (suele estar a la derecha junto al Explorador de soluciones):

1. **Commit Local:** Escriba un mensaje descriptivo en el cuadro de texto (ej: *"Se agregó validación de precios"*).
2. **Confirmar todo:** Haga clic en el botón **Confirmar todo** (Commit All).
3. **Sincronizar:** Haga clic en la flecha que apunta hacia arriba o en el botón **Insertar** (Push) para mandar esos cambios a GitHub.

Una estructura profesional para su archivo **`README.md`**. Este diseño está pensado para que cualquier persona (o un reclutador) que visite su repositorio de GitHub entienda inmediatamente qué hace el proyecto, cómo está estructurada la base de datos y cómo ejecutarlo.

Cree un archivo llamado exactamente **`README.md`** en la raíz de su proyecto dentro de Visual Studio, pegue el siguiente contenido y personalice los textos entre corchetes `[ ]`:

```
# 🛒 Sistema de Gestión de Tienda (CRUD Multitabla)

¡Bienvenido! Este es un proyecto de escritorio desarrollado en **C# con Windows Forms** utilizando **Visual Studio 2022**. Implementa un sistema CRUD (Crear, Leer, Actualizar, Eliminar) completo y multitabla conectado a una base de datos local de SQL Server (`.mdf`).

## 🚀 Características del Proyecto
- **CRUD Completo:** Gestión integral de registros (Inserción, lectura, actualización y borrado lógico/físico).
- **Arquitectura Multitabla:** Manejo de relaciones Uno a Muchos (1:N) entre Categorías y Productos.
- **ORM Moderno:** Utiliza **Entity Framework Core** para el mapeo de datos, eliminando la necesidad de escribir consultas SQL manuales.
```

```
- **Interfaz Intuitiva:** Grid dinámico con actualización en tiempo real y
controles autolimpiables.

---

## Arquitectura de la Base de Datos

El sistema utiliza una base de datos local basada en servicios (`TiendaDB.mdf`)
con la siguiente estructura de tablas y dependencias:

    CATEGORIAS {
        int Id PK
        string Nombre
    }

    PRODUCTOS {
        int Id PK
        string Nombre
        decimal Precio
        int CategoriaId FK
    }

### Script de Creación (DDL)

-- Tabla: Categorias
CREATE TABLE [dbo].[Categorias] (
    [Id]          INT           IDENTITY (1, 1) NOT NULL,
    [Nombre]      NVARCHAR (50) NOT NULL,
    PRIMARY KEY CLUSTERED ([Id] ASC)
);

-- Tabla: Productos
CREATE TABLE [dbo].[Productos] (
    [Id]          INT            IDENTITY (1, 1) NOT NULL,
    [Nombre]      NVARCHAR (100) NOT NULL,
    [Precio]      DECIMAL (18, 2) NOT NULL,
    [CategoriaId] INT            NOT NULL,
    PRIMARY KEY CLUSTERED ([Id] ASC),
    CONSTRAINT [FK_Productos_Categorias] FOREIGN KEY ([CategoriaId]) REFERENCES
[dbo].[Categorias] ([Id]) ON DELETE CASCADE
);

---
```

```
## Tecnologías y Herramientas Herramientas Utilizadas
- **Lenguaje:** C# (.NET [Indica tu versión, ej: 8.0])
- **Entorno:** Visual Studio 2022
- **Tecnología de UI:** Windows Forms (WinForms)
- **Base de Datos:** SQL Server Express LocalDB (`.mdf`)
- **ORM:** Entity Framework Core (Microsoft.EntityFrameworkCore.SqlServer)

---

## Requisitos e Instalación

Para clonar y ejecutar esta aplicación localmente, necesita tener instalado
**Visual Studio 2022** con la carga de trabajo **Desarrollo de escritorio de
.NET** y el componente **LocalDB de SQL Server Express**.

1. **Clone el repositorio:**
   git clone https://github.com[SuUsuarioDeGitHub]/[NombreDeSuRepositorio].git
2. Abra el archivo de solución `.sln` en Visual Studio 2022.
3. Asegúrese de restaurar los paquetes NuGet si el IDE no lo hace
automáticamente.
4. Presione `F5` o haga clic en **Iniciar** para ejecutar el proyecto.

> **Nota sobre la Base de Datos:** El proyecto está configurado para buscar el
archivo `TiendaDB.mdf` automáticamente en el directorio de salida. Asegúrese de
que el archivo `.mdf` tenga la propiedad *Copiar en el directorio de salida*
configurada como *Copiar si es posterior*.
```

¿Cómo guardarlo y subirlo a GitHub?

1. En Visual Studio, haga clic derecho sobre la Solución -> **Agregar** -> **Nuevo elemento...**
2. Busca **Archivo de texto**, nomúbrelo exactamente **`README.md`** (asegúrese de borrar la extensión `.txt`) y dele a Agregar.
3. Pegue el código de arriba, edite los campos que están en corchetes y guarde el archivo (`Ctrl + S`).
4. Abra la pestaña **Cambios de Git**, escriba un mensaje como *"Se agrega archivo README profesional"* y haga clic en **Confirmar todo y push** para enviarlo a GitHub.

GENERANDO INFORMES EN WINDOWS FORMS (DESKTOP)

Para generar un informe o archivo **PDF** con múltiples tablas relacionadas en **Visual Studio 2022** y **Windows Forms**, la mejor alternativa nativa y gratuita (sin depender de librerías de pago complejas) es utilizar Microsoft Reporting Services (RDLC). A continuación, se muestra el paso a paso detallado para estructurar su reporte con las tablas relacionales de `Categorias` y `Productos` que definimos anteriormente.



1. Instalar la extensión y paquetes necesarios

Para poder diseñar reportes en Visual Studio 2022 necesita instalar la herramienta de diseño:

1. Vaya al menú superior **Extensiones** -> **Administrar extensiones**.
2. Busque e instale: **Microsoft RDLC Report Designer 2022**. (Si se lo pide, reinicie el Visual Studio).
3. Abra su proyecto en Windows Forms y vaya a **Herramientas** -> **Administrador de paquetes NuGet** -> **Consola del Administrador de paquetes**.
4. Instale el visor de reportes para Windows Forms ejecutando:

```
Install-Package
Microsoft.ReportingServices.ReportViewerControl.Winforms
```

2. Crear el Conjunto de Datos (Dataset) del Reporte

El reporte necesita conocer la estructura de los datos que va a pintar.

1. Haga clic derecho en su proyecto -> **Agregar** -> **Nuevo elemento**.
2. Elija **Conjunto de datos** (Dataset) y llámelo `TiendaDataSet.xsd`.
3. Se abrirá un lienzo en blanco. Haga clic derecho en el lienzo -> **Agregar** -> **DataTable**.
4. Llámelo `dtProductosReporte`.
5. Agregue las columnas haciendo clic derecho sobre la tabla -> **Agregar** -> **Columna** (o presiona `Ctrl + L`):
    - `Id` (System.Int32)
    - `Nombre` (System.String)
    - `Precio` (System.Decimal)
    - `Categoria` (System.String) *(Aquí guardaremos el nombre de la tabla relacionada)*

3. Diseñar el Reporte (.rdlc)

1. Haga clic derecho en su proyecto -> **Agregar** -> **Nuevo elemento**.
2. Elija **Informe** (Report) y llámalo `InformeProductos.rdlc`.
3. En los controles del reporte (si no lo ve, vaya al menú a *Ver -> Datos del informe*):
    - Haga clic derecho en **Conjuntos de datos** -> **Agregar conjunto de datos**.
    - Póngale de nombre `DataSetProductos`.
    - En *Origen de datos*, seleccione `TiendaDataSet`.
    - En *Conjunto de datos disponible*, seleccione `dtProductosReporte` y clic en **Aceptar**.
4. **Diseñar el contenido:**
    - Haga clic derecho en el lienzo blanco del informe -> **Insertar** -> **Tabla**.
    - Arrastre los campos desde la ventana *Datos del informe* hacia las celdas de la fila "Datos" de su tabla (por ejemplo: `Nombre`, `Precio`, `Categoria`).

Para agrupar por Categoría (Múltiples Tablas Visuales): Si quiere que se vea una tabla limpia dividida por categorías, haga clic derecho en la fila de datos de la tabla del reporte -> **Grupo de filas** -> **Propiedades del grupo** y agrúpelo por el campo `Categoria`. Esto creará subtotales o separadores automáticos por cada tabla relacional.

4. Preparar los Datos en el Controlador (`ProductoController.cs`)

Necesitamos un método que devuelva exactamente la información combinada que pide nuestro Dataset:

```
public DataTable ObtenerDatosParaReporte()
{
    DataTable dt = new DataTable();
    using (SqlConnection conn = DbContext.GetConnection())
    {
        // El INNER JOIN junta las múltiples tablas de la BD en un solo resultado plano
        string query = @"SELECT p.Id, p.Nombre, p.Precio, c.Nombre AS Categoria
                         FROM Productos p
                         INNER JOIN Categorias c ON p.CategoriaId = c.Id";
        using (SqlCommand cmd = new SqlCommand(query, conn))
        {
            using (SqlDataAdapter adapter = new SqlDataAdapter(cmd))
            {
                adapter.Fill(dt);
            }
        }
    }
    return dt;
}
```

5. Generar y Exportar el PDF directamente desde código

Puede añadir un botón llamado `btnExportarPDF` en su formulario principal. Al hacer clic, procesará el reporte en segundo plano y **guardará el archivo PDF directamente en el disco del usuario** sin necesidad de mostrar ventanas emergentes molestas.

```
using System;
using System.Data;
using System.IO;
using System.Windows.Forms;
using Microsoft.Reporting.WinForms; // Asegúrese de añadir esta referencia

private void btnExportarPDF_Click(object sender, EventArgs e)
{
    try
    {
        // 1. Obtener los datos combinados de las tablas
        ProductoController controller = new ProductoController();
        DataTable datos = controller.ObtenerDatosParaReporte();

        // 2. Configurar el ReportViewer en memoria (sin controles visuales)
        LocalReport reporteLocal = new LocalReport();
```

```
        // Indicar la ruta del archivo de diseño del informe
        // Nota: Asegúrate de que las propiedades del archivo .rdlc tengan
'Copiar en el directorio de salida = Copiar si es posterior'
        reporteLocal.ReportPath     =     Path.Combine(Application.StartupPath,
"InformeProductos.rdlc");

        // Vincular el DataTable al Dataset del informe RDLC
        ReportDataSource rds = new ReportDataSource("DataSetProductos", datos);
        reporteLocal.DataSources.Clear();
        reporteLocal.DataSources.Add(rds);

        // 3. Configurar parámetros de renderizado a PDF
        string reportType = "PDF";
        string mimeType;
        string encoding;
        string fileNameExtension;
        Warning[] warnings;
        string[] streams;

        // 4. Renderizar el reporte a un arreglo de bytes
        byte[] bytesPdf = reporteLocal.Render(
            reportType,
            null,
            out mimeType,
            out encoding,
            out fileNameExtension,
            out streams,
            out warnings
        );

        // 5. Guardar el archivo mediante un cuadro de diálogo para el usuario
        SaveFileDialog saveFileDialog = new SaveFileDialog();
        saveFileDialog.Filter = "Archivos PDF (*.pdf)|*.pdf";
        saveFileDialog.FileName = "Reporte_Productos_y_Categorias.pdf";

        if (saveFileDialog.ShowDialog() == DialogResult.OK)
        {
            File.WriteAllBytes(saveFileDialog.FileName, bytesPdf);
            MessageBox.Show("¡PDF generado y guardado con éxito!", "Éxito",
MessageBoxButtons.OK, MessageBoxIcon.Information);
        }
    }
```

```
    catch (Exception ex)
    {
        MessageBox.Show("Error al generar el PDF: " + ex.Message, "Error",
MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}
```

VISTA PREVIA DE INFORMES EN VISUAL STUDIO

Mostrar una vista previa interactiva antes de imprimir o guardar el archivo, utilizaremos el control visual ReportViewer directamente en un formulario de Windows Forms. A continuación, se muestran los pasos para integrar el visor y conectarlo con los datos de sus múltiples tablas.

1. Preparar el Formulario del Visor (`FormReporte.cs`)

Lo ideal es crear un formulario independiente dedicado exclusivamente a mostrar el reporte.

1. Haga clic derecho en su proyecto -> **Agregar** -> **Formulario (Windows Forms)**.
2. Nómbrelo como `FormReporte.cs`.
3. Haga el formulario más grande (por ejemplo, `Width: 800`, `Height: 600`) para que el reporte se lea cómodamente.
4. Abra el **Cuadro de herramientas**, buque el control llamado **ReportViewer** y arrástrelo al formulario. *(Si no le aparece en el cuadro de herramientas tras instalar el paquete NuGet, compile el proyecto presionando `Ctrl + Mayús + B`).*
5. Seleccione el control `reportViewer1`, busque su propiedad **Dock** en la ventana de propiedades y cámbiela a **Fill** para que ocupe todo el formulario automáticamente.

2. Código del Formulario del Reporte (`FormReporte.cs`)

Modificaremos el constructor del nuevo formulario para que reciba los datos desde el formulario principal y los cargue en el visor de manera limpia:

```
using System;
using System.Data;
using System.IO;
using System.Windows.Forms;
using Microsoft.Reporting.WinForms;

public partial class FormReporte : Form
{
    // Variable para recibir los datos combinados de las tablas
    private DataTable _datosReporte;

    // Modificamos el constructor para inyectar los datos
    public FormReporte(DataTable datos)
    {
        InitializeComponent();
        this._datosReporte = datos;
    }
```

```
    private void FormReporte_Load(object sender, EventArgs e)
    {
        try
        {
            // 1. Limpiar configuraciones previas del visor
            reportViewer1.LocalReport.DataSources.Clear();

            // 2. Definir la ruta del archivo de diseño .rdlc
            string rutaReporte = Path.Combine(Application.StartupPath,
"InformeProductos.rdlc");
            reportViewer1.LocalReport.ReportPath = rutaReporte;

            // 3. Vincular los datos de las tablas al origen de datos del RDLC
            // "DataSetProductos" debe coincidir exactamente con el nombre que
pusiste en el .rdlc
            ReportDataSource rds = new ReportDataSource("DataSetProductos",
_datosReporte);
            reportViewer1.LocalReport.DataSources.Add(rds);

            // 4. Procesar y renderizar el reporte de forma visual
            reportViewer1.RefreshReport();
        }
        catch (Exception ex)
        {
            MessageBox.Show("Error al cargar la vista previa: " + ex.Message,
"Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
    }
}
```

3. Invocar la Vista Previa desde tu Formulario Principal (`Form1.cs`)

En su formulario de gestión (donde tiene el botón de guardar, editar y la lista de productos), añada un nuevo botón llamado `btnVistaPrevia` y programe su evento clic:

```
private void btnVistaPrevia_Click(object sender, EventArgs e)
{
    // 1. Instanciar el controlador y obtener los datos de múltiples tablas
(INNER JOIN)
    ProductoController controller = new ProductoController();
    DataTable datos = controller.ObtenerDatosParaReporte();
```

```
    if (datos != null && datos.Rows.Count > 0)
    {
        // 2. Crear el formulario del reporte pasando los datos por el constructor
        FormReporte visor = new FormReporte(datos);

        // 3. Mostrar el formulario como una ventana emergente modal
        visor.ShowDialog();
    }
    else
    {
        MessageBox.Show("No hay datos disponibles para mostrar en el reporte.",
"Aviso", MessageBoxButtons.OK, MessageBoxIcon.Warning);
    }
}
```

Ventajas de usar ReportViewer en la interfaz:

- **Exportación Nativa:** El usuario tendrá una barra de herramientas en la parte superior del reporte donde podrá hacer clic en el ícono de guardar (disquete) y elegir exportar de forma nativa a **PDF**, **Excel** o **Word** sin código adicional.
- **Impresión Directa:** Incluye un botón para configurar la página y mandar el documento directamente a la impresora física.
- **Buscador:** Cuenta con una barra de búsqueda integrada para localizar palabras o datos específicos dentro del informe.

FILTROS DINAMICOS EN INFORMES

Para implementar **filtros dinámicos**, permitiremos que el usuario filtre el reporte desde el formulario principal por una **Categoría** específica (usando el `ComboBox`) y por un **Rango de Precios** (usando dos `TextBox`). A continuación, se muestra cómo modificar la consulta SQL, cómo pasar **parámetros de texto** al reporte (para mostrar qué filtros se aplicaron en el encabezado) y cómo conectar todo en el formulario.

1. Modificar el Controlador para Consultas Filtradas (`ProductoController.cs`)

Modificaremos nuestro método de obtención de datos para que reciba parámetros opcionales y construya la consulta de forma dinámica para evitar inyecciones SQL:

```
public DataTable ObtenerDatosParaReporteFiltrado(int? categoriaId, decimal? precioMin, decimal? precioMax)
{
    DataTable dt = new DataTable();
    using (SqlConnection conn = DbContext.GetConnection())
    {
        // Base de la consulta con JOIN para traer las múltiples tablas
        string query = @"SELECT p.Id, p.Nombre, p.Precio, c.Nombre AS Categoria
                         FROM Productos p
                         INNER JOIN Categorias c ON p.CategoriaId = c.Id
                         WHERE 1=1"; // 1=1 facilita concatenar condiciones AND

        // Construcción dinámica de filtros
        if (categoriaId.HasValue && categoriaId.Value > 0)
        {
            query += " AND p.CategoriaId = @categoriaId";
        }
        if (precioMin.HasValue)
        {
            query += " AND p.Precio >= @precioMin";
        }
        if (precioMax.HasValue)
        {
            query += " AND p.Precio <= @precioMax";
        }

        using (SqlCommand cmd = new SqlCommand(query, conn))
        {
            // Asignar parámetros solo si se utilizan
```

```
            if (categoriaId.HasValue && categoriaId.Value > 0)
                cmd.Parameters.AddWithValue("@categoriaId", categoriaId.Value);

            if (precioMin.HasValue)
                cmd.Parameters.AddWithValue("@precioMin", precioMin.Value);

            if (precioMax.HasValue)
                cmd.Parameters.AddWithValue("@precioMax", precioMax.Value);

            using (SqlDataAdapter adapter = new SqlDataAdapter(cmd))
            {
                adapter.Fill(dt);
            }
        }
    }
    return dt;
}
```

2. Agregar Parámetros en el Reporte (`InformeProductos.rdlc`)

Es una buena práctica que el PDF o reporte muestre qué filtros se aplicaron (por ejemplo: *"Filtro: Tecnología | Precios entre $10 y $100"*).

1. Abra su archivo `InformeProductos.rdlc` en el diseñador de Visual Studio.
2. En la ventana **Datos del informe** (lado izquierdo), haga clic derecho en **Parámetros** -> **Agregar parámetro**.
3. Cree un parámetro llamado `FiltroUsuario` de tipo **Texto** (String) y presione Aceptar.
4. Arrastre este nuevo parámetro desde la lista de datos del informe hacia el encabezado o la parte superior de su lienzo del reporte para que se imprima textualmente.

3. Modificar el Constructor del Visor (`FormReporte.cs`)

Ahora el formulario del reporte debe aceptar tanto el `DataTable` como la cadena de texto con la descripción de los filtros:

```
using System;
using System.Data;
using System.IO;
using System.Windows.Forms;
using Microsoft.Reporting.WinForms;

public partial class FormReporte : Form
{
    private DataTable _datosReporte;
```

```
    private string _descripcionFiltro;

    // Modificamos el constructor para recibir los datos y el texto del filtro
    public FormReporte(DataTable datos, string descripcionFiltro)
    {
        InitializeComponent();
        this._datosReporte = datos;
        this._descripcionFiltro = descripcionFiltro;
    }

    private void FormReporte_Load(object sender, EventArgs e)
    {
        try
        {
            reportViewer1.LocalReport.DataSources.Clear();

          string rutaReporte = Path.Combine(Application.StartupPath, "InformeProductos.rdlc");
            reportViewer1.LocalReport.ReportPath = rutaReporte;

            // 1. Enviar los parámetros definidos en el RDLC
            ReportParameter[] parametros = new ReportParameter[1];
            parametros[0] = new ReportParameter("FiltroUsuario", _descripcionFiltro);
            reportViewer1.LocalReport.SetParameters(parametros);

            // 2. Enviar el DataTable
           ReportDataSource rds = new ReportDataSource("DataSetProductos", _datosReporte);
            reportViewer1.LocalReport.DataSources.Add(rds);

            reportViewer1.RefreshReport();
        }
        catch (Exception ex)
        {
            MessageBox.Show("Error al cargar el reporte filtrado: " + ex.Message,
"Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
    }
}
```

4. Invocar con Filtros desde el Formulario Principal (`Form1.cs`)

En su formulario de diseño, añada dos campos de texto (`txtFiltroMin`, `txtFiltroMax`) para los rangos de precio. Cuando el usuario presione el botón de generar reporte, capturaremos las variables para enviarlas al controlador:

```
private void btnReporteFiltrado_Click(object sender, EventArgs e)
{
    // 1. Capturar filtros de la interfaz de usuario de forma segura
    int? categoriaId = null;
    if (cmbCategorias.SelectedValue != null)
    {
        categoriaId = Convert.ToInt32(cmbCategorias.SelectedValue);
    }

    decimal? precioMin = null;
    if (!string.IsNullOrEmpty(txtFiltroMin.Text) &&
decimal.TryParse(txtFiltroMin.Text, out decimal pMin))
    {
        precioMin = pMin;
    }

    decimal? precioMax = null;
    if (!string.IsNullOrEmpty(txtFiltroMax.Text) &&
decimal.TryParse(txtFiltroMax.Text, out decimal pMax))
    {
        precioMax = pMax;
    }

    // 2. Construir la cadena de texto descriptiva para el encabezado del reporte
    string nombreCategoria = cmbCategorias.SelectedIndex >= 0 ?
cmbCategorias.Text : "Todas";
    string txtMinInfo = precioMin.HasValue ? $"${precioMin.Value}" : "Mínimo";
    string txtMaxInfo = precioMax.HasValue ? $"${precioMax.Value}" : "Máximo";
    string descripcionFiltros = $"Categoría: {nombreCategoria} | Rango Precios:
({txtMinInfo} - {txtMaxInfo})";

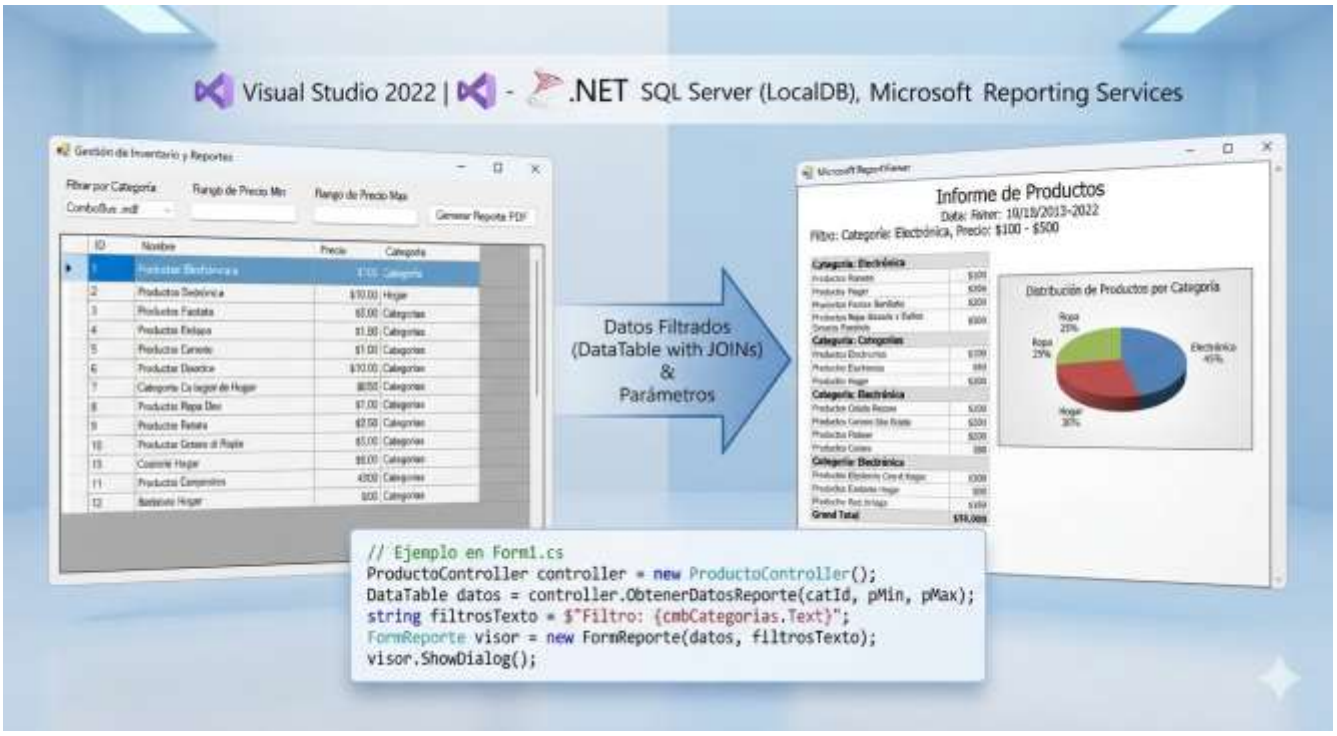
    // 3. Consultar la base de datos con los filtros dinámicos aplicados
    ProductoController controller = new ProductoController();
    DataTable datosFiltrados =
controller.ObtenerDatosParaReporteFiltrado(categoriaId, precioMin, precioMax);

    if (datosFiltrados != null && datosFiltrados.Rows.Count > 0)
    {
        // 4. Abrir la vista previa pasando los datos y la etiqueta del filtro
        FormReporte visor = new FormReporte(datosFiltrados, descripcionFiltros);
        visor.ShowDialog();
    }
    else
```

```
    {
        MessageBox.Show("No se encontraron productos que coincidan con los filtros seleccionados.", "Sin Resultados", MessageBoxButtons.OK, MessageBoxIcon.Information);
    }
}
```

GRÁFICOS EN INFORMES

Para agregar un **gráfico estadístico** (como un gráfico de barras o de pastel) dentro del mismo reporte de **ReportViewer (RDLC)**, necesitamos procesar la información de las tablas relacionales de modo que el motor de reportes pueda agruparla y contarla automáticamente.
A continuación, se muestra el paso a paso para añadir un gráfico que muestre la **cantidad de productos por categoría** o la **suma de precios por categoría** visualmente.



1. Diseñar el Gráfico en el Reporte (`InformeProductos.rdlc`)

1. Abra el archivo **InformeProductos.rdlc** en el diseñador de Visual Studio.
2. Haga clic derecho sobre un espacio en blanco del lienzo del reporte -> **Insertar** -> **Gráfico...** (Chart).
3. Seleccione el tipo de gráfico que prefiera. Para este escenario de categorías, el gráfico de **Columnas (Barras verticales)** o **Circular (Pastel)** funciona mejor. Presione *Aceptar*.
4. Arrastre y acomode el gráfico en la posición del reporte donde desee que aparezca (por ejemplo, en el encabezado o al final de la tabla de datos).

2. Configurar los Datos del Gráfico

Al hacer un solo clic sobre el gráfico que acaba de insertar, se abrirá un panel flotante a la derecha llamado **Datos del gráfico**. Configúrelo arrastrando los campos de su `DataSetProductos` de la siguiente manera:

- **Valores (Values):** Haga clic en el botón + y seleccione el campo `Nombre` (o `Id`). Automáticamente cambiará a **`Count(Nombre)`**, lo que significa que contará cuántos productos existen en cada sección.
- **Grupos de categorías (Category Groups):** Haga clic en el botón + y selecciona el campo `Categoria`. Esto asegurará que cada barra vertical corresponda a una categoría distinta de su tabla relacional.
- **Grupos de series (Series Groups):** Déjelo en blanco (a menos que quiera subcategorías).

Si desea que el gráfico muestre el dinero acumulado en lugar de la cantidad de productos, en ***Valores*** *seleccione el campo* `Precio` *y se configurará automáticamente como* ***`Sum(Precio)`.***

3. Habilitar Etiquetas de Datos (Opcional pero Recomendado)

Para que el usuario pueda ver el número exacto encima de cada barra sin tener que adivinarlo por la escala del eje:

1. En el diseñador del reporte, haga clic derecho sobre cualquiera de las barras del gráfico de ejemplo.
2. Seleccione la opción **Mostrar etiquetas de datos** (Show Data Labels).
3. Aparecerán números marcadores estáticos (`#PERCENT` o los valores reales) encima de sus barras.

4. Ejecución Automática

¡No necesita modificar su código de C#!

Como el método **`ObtenerDatosParaReporteFiltrado`** en **`ProductoController.cs`** ya recupera la información combinada de sus múltiples tablas (`Productos` y `Categorias`), el componente de gráficos de **Reporting Services** se encargará de recalcular y redibujar las barras dinámicamente basándose exclusivamente en los datos filtrados que le envíe en el `DataTable`. Cuando el usuario filtre por ejemplo para ver solo productos de "Línea Blanca" o de un rango de precios específico, el gráfico se actualizará en tiempo real mostrando únicamente las categorías que entraron en ese filtro.